

TECHNICAL REFERENCE

Gemini Live in the AI Interviewer

How real-time voice, transcription, and post-interview scoring are wired up using Google's Gemini Live API — architecture, session config, prompts, and the client/server data flow.

Project: ai-interviewer-main · Model: gemini-2.5-flash-native-audio-latest
Generated 2026-09-06

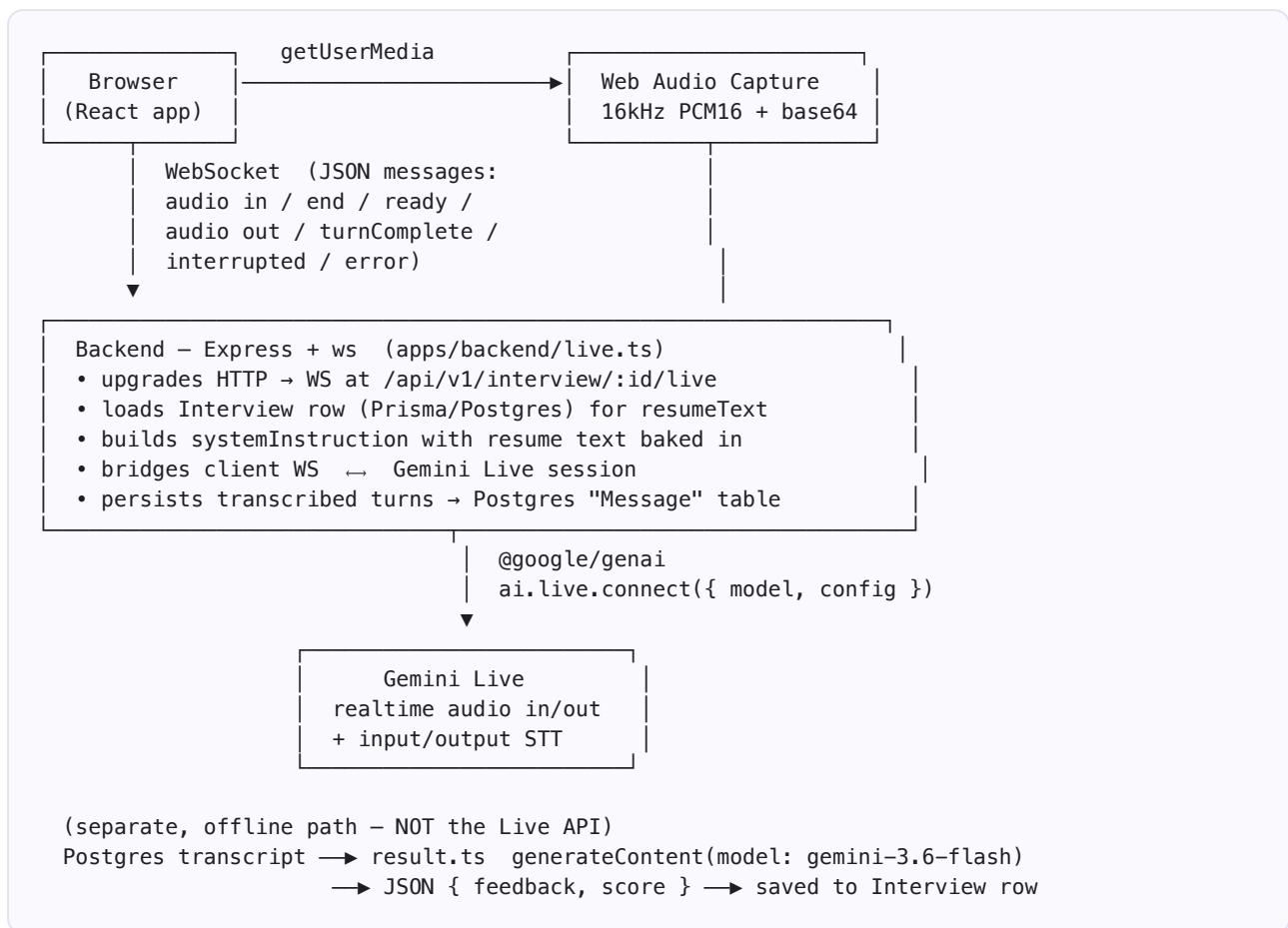
1 Overview

The app runs a **voice-only** AI interview. The candidate speaks into the browser; audio streams to a Node/Bun backend over a plain WebSocket; the backend bridges that audio to **Gemini Live** using Google's `@google/genai` SDK. Gemini speaks back in real time, transcribes both sides of the conversation, and the transcript is persisted to Postgres. After the interview ends, a *separate*, non-Live Gemini call scores the transcript and returns structured JSON feedback.

Two distinct Gemini touchpoints exist:

Touchpoint	API	Model	Purpose
Live interview session	<code>ai.live.connect()</code>	<code>gemini-2.5-flash-native-audio-latest</code>	Real-time duplex voice conversation + transcription
Post-interview scoring	<code>ai.models.generateContent()</code>	<code>gemini-3.6-flash</code>	One-shot JSON score/feedback from saved transcript

2 Architecture



Key property: the client **never** talks to Gemini directly. The backend is the only holder of `GEMINI_API_KEY`, and it purely relays audio + persists the transcript — it does no buffering, batching, or VAD of its own. Turn-taking and voice activity detection are entirely handled by Gemini Live's defaults.

3 SDK & Dependencies

Package	Version	Role
@google/genai	^2.8.0	Official unified GenAI SDK — exposes both <code>ai.live.connect()</code> and <code>ai.models.generateContent()</code>
ws	^8.20.1	Client ↔ backend WebSocket only (not used toward Gemini — the SDK manages that connection internally)
pdf-parse	—	Extracts resume text from uploaded PDF, fed into the system instruction
zod-to-json-schema	—	Builds the JSON schema used to constrain the post-interview scoring output

The legacy `@google/generative-ai` SDK is **not** used. The frontend has no Gemini dependency at all — every Gemini call is server-side.

4 Environment Variables

Variable	Used?	Purpose
GEMINI_API_KEY	✅ Yes	Auth for <code>@google/genai</code> client (both Live + generateContent)
DATABASE_URL	✅ Yes	Postgres connection string (Prisma)
DEEPGRAM_API_KEY	⚠️ Unused	Present in <code>.env</code> , no references found in code
OPENAI_KEY	⚠️ Unused	Present in <code>.env</code> , no references found in code

5 Live Session Configuration

apps/backend/live.ts

```
const session = await ai.live.connect({
  model: "gemini-2.5-flash-native-audio-latest",
  config: {
    responseModalities: [Modality.AUDIO],
    inputAudioTranscription: {},
    outputAudioTranscription: {},
    systemInstruction,
  },
});
```

- **AUDIO only** Response modality is audio-only — no text modality requested.
- **STT on** Both `inputAudioTranscription` and `outputAudioTranscription` are enabled (empty options = defaults, no language hints) — this is what makes text transcript persistence possible.
- **Default voice** No `speechConfig` / `voiceConfig` set — uses Gemini's default voice.
- **Default VAD** No `realtimeInputConfig.automaticActivityDetection` override — relies fully on Gemini Live's built-in turn detection. The only turn-taking signal surfaced to the client is the `interrupted` event (barge-in).
- **Default params** No `temperature`, `topP`, `topK`, or `maxOutputTokens` set.
- Input audio to Gemini: `audio/pcm;rate=16000` (16kHz mono PCM16, downsampled client-side).
- Output audio from Gemini: assumed 24kHz mono PCM16 (per an in-code comment flagging it as unverified at runtime).

6 System Prompts

Interview persona (Live session)

[apps/backend/live.ts](#)

```
const systemInstruction = `You are supposed to interview this user on their computer science background. Ask around 2-3 questions based on their experience. Please use english only during the interview. Here is the candidate's resume, will give you a rough idea about what the user does -  
## Resume  
${interview.resumeText}  
`;
```

The resume text is extracted server-side from an uploaded PDF ([pdf-parse](#) , in [apps/backend/resume.ts](#)) and interpolated directly into the prompt at session start, giving Gemini candidate context before the first question.

Post-interview scoring (separate, non-Live call)

[apps/backend/result.ts](#)

```
const RESULT_PROMPT = `  
You are an expert evaluator. Your job is to evaluate the users interview. Give them a score out of 10 and also let them know any feedback you have about thier interview.  
Please return only a json which looks like this -  
{ feedback: string, score: number }  
DO NOT RETURN ANY OTHER TEXT  
${USER_TRANSCRIPT}  
`;
```

Called via [generateContent](#) with [responseMimeType: "application/json"](#) and a [responseJsonSchema](#) built from a Zod schema — structured output, not the Live API.

7 Client ↔ Backend Message Protocol

Direction	Message	Meaning
Client → Backend	<code>{type:"audio", data}</code>	Base64 16kHz PCM16 mono chunk
Client → Backend	<code>{type:"end"}</code>	End of candidate's audio stream
Backend → Client	<code>{type:"ready"}</code>	Gemini Live session is open; unlocks mic sending
Backend → Client	<code>{type:"audio", data}</code>	Base64 24kHz PCM16 model speech
Backend → Client	<code>{type:"turnComplete"}</code>	Model finished its turn
Backend → Client	<code>{type:"interrupted"}</code>	Barge-in — stop playback immediately
Backend → Client	<code>{type:"error", message}</code>	Error surfaced from the session

Backend → Gemini: `session.sendRealtimeInput({ audio: { data, mimeType: "audio/pcm;rate=16000" } })`, then `audioStreamEnd: true` on end. Gemini → Backend arrives as `LiveServerMessage` callbacks carrying audio chunks, `serverContent.inputTranscription` / `outputTranscription`, `turnComplete`, and `interrupted`.

Transcribed text is buffered per speaker and flushed to the Postgres `Message` table (`interviewId`, `type: User|Assistant`, `message`) whenever a segment finishes, or on `turnComplete` /socket close.

8 Frontend Audio Pipeline

No WebRTC — plain WebSocket + Web Audio API. Files: `Interview.tsx`, `lib/audio.ts`, `lib/pcmPlayer.ts`, `components/VoiceOrb.tsx`.

- **Capture:** `getUserMedia({audio:true})` → `MediaStreamSource` → `ScriptProcessorNode` (4096 buffer) → downsample to 16kHz → base64 → sent over WS. Gated by a `micReadyRef` flag that only flips once `{type:"ready"}` arrives.
- **Playback:** incoming base64 PCM16 @ 24kHz decoded to Float32, wrapped in `AudioBuffer` s, scheduled back-to-back via `nextStartTime` bookkeeping through a `GainNode` for gapless playback.
- **Barge-in:** on `{type:"interrupted"}`, all scheduled-but-unplayed buffer sources are stopped immediately.
- **Level metering:** an `AnalyserNode` tapped off both mic and playback drives RMS "who's speaking" UI (`VoiceOrb`) — local UI only, never sent to Gemini.
- **Connection URL:** `ws://localhost:3001/api/v1/interview/:interviewId/live` (backend URL currently hardcoded, no env override).

9 What's *not* used

- No video — despite "voice/video" framing, this is audio-only.
- No tools / function calling in the Live session — no "next question" or "score answer" tool; scoring happens entirely out-of-band after the interview ends.
- No direct client → Gemini connection — the backend proxy pattern keeps the API key server-side.
- No explicit voice selection, VAD tuning, or generation parameters — all Gemini defaults.

10 Key Files Reference

File	Role
<code>apps/backend/live.ts</code>	Live API session, system prompt, session config, WS bridge
<code>apps/backend/result.ts</code>	Post-interview scoring (non-Live generateContent, JSON schema output)
<code>apps/backend/index.ts</code>	Express routes: resume upload → interview creation, result endpoint
<code>apps/backend/resume.ts</code>	PDF resume text extraction
<code>apps/backend/db.ts</code>	Prisma client (Postgres)
<code>apps/backend/prisma/schema.prisma</code>	Interview / Message data model
<code>apps/backend/package.json</code>	@google/genai@^2.8.0, ws@^8.20.1
<code>apps/frontend/src/components/Interview.tsx</code>	Client call flow, WS wiring, level meters
<code>apps/frontend/src/lib/audio.ts</code>	PCM downsample/encode helpers
<code>apps/frontend/src/lib/pcmPlayer.ts</code>	Gapless playback + interrupt handling
<code>apps/frontend/src/components/VoiceOrb.tsx</code>	Speaking-indicator UI
<code>apps/frontend/src/lib/config.ts</code>	BACKEND_URL